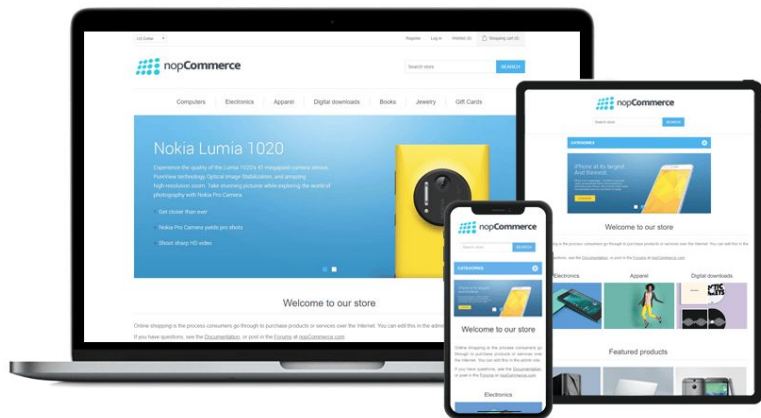


Architectural Evolution of nopCommerce

David Pelicano ◉ Henrique Ferreira ◉ João Monteiro ◉ Jorge Domingues

05.05.2026



Index

A

Scenario C

B

Current State Analysis

C

Domain & Boundary Model

D

Quality Attributes Scenarios

E

Framework: ADD

F

Iterations

G

Risk & Validation Plan

H

Feasibility Spike

I

Conclusion

Scenario C: Omnichannel Commerce Core

Evolving nopCommerce under real architectural pressure

VerdeMart: web storefront + warehouse + physical store + carrier

Each system runs independently and can fail on its own

The challenge: keep the commerce core useful when surrounding systems degrade

Why this scenario: reliability, availability, and consistency failures have direct business impact

Use Cases

Use Case 1:

Buy Online, Fulfill Through Another Channel

Use Case 2:

Cross-Channel Order-State Visibility

Mandatory Pressure Point

Commerce core remains operational when surrounding systems are unavailable or slow

Current State Analysis

- In-process events — no delivery guarantee, no retry, no persistence
- No inventory lock — concurrent orders can oversell the same unit
- No integration pattern — every external connection built from scratch



Domain & Boundary Model

Subdomains

- Core: Order Management, Inventory Allocation, Fulfillment Execution
- Supporting: Product Catalog, Payments, Shipping
- Generic: Identity, Search (out of scope)

Bounded Contexts

- nopCommerce — order lifecycle, customer experience
- OpenBoxes — physical stock, pick/pack/ship
- POS — in-store sales channel
- WireMock — carrier

Ownership

- Order lifecycle → nopCommerce
- Physical stock truth → OpenBoxes

Quality Attributes Scenarios

QAS-1: Reliability

Zero orders lost during OpenBoxes outage

QAS-2: Consistency

Zero oversell under concurrent web + POS load

QAS-3: Availability

Checkout under 3s regardless of surrounding system latency

QAS-4: Recoverability

Missed events must self-heal after outage

QAS-5: Visibility

External state changes must reach the customer quickly

Framework: Attribute-Driven Design (ADD)

Why ADD?

- Scenario is quality-attribute-driven, not feature-driven
- Every decision traces back to a QAS
- Lean enough for this project's scope

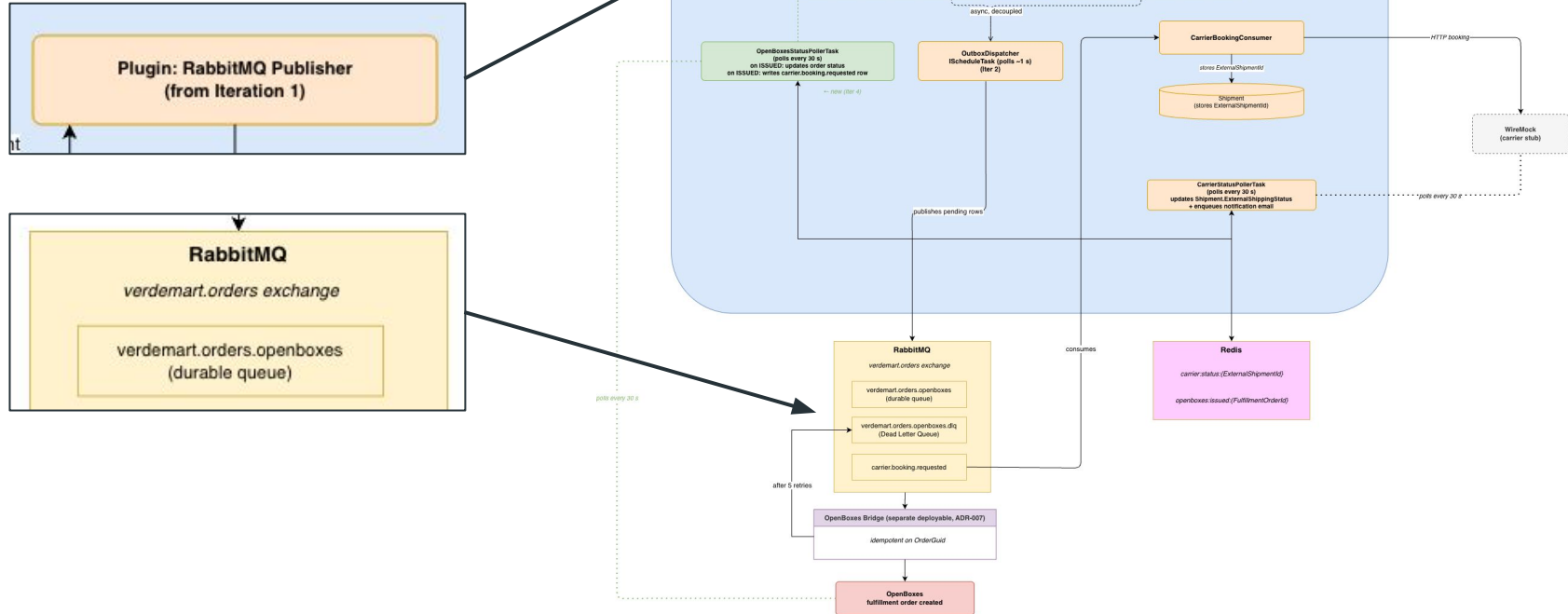
Step	Question
1. Review inputs	what drivers, constraints, and requirements do we have?
2. Pick iteration goal	what problem are we solving now?
3. Choose element to refine	where will we decompose?
4. Select design concepts	what patterns, tactics, or external services fit?
5. Instantiate elements	what responsibilities and interfaces appear?
6. Sketch views and decisions	how will we communicate the result?
7. Analyze current design	did the iteration satisfy its drivers?

Iteration 1

QAS-1: Reliability Zero orders lost during OpenBoxes outage

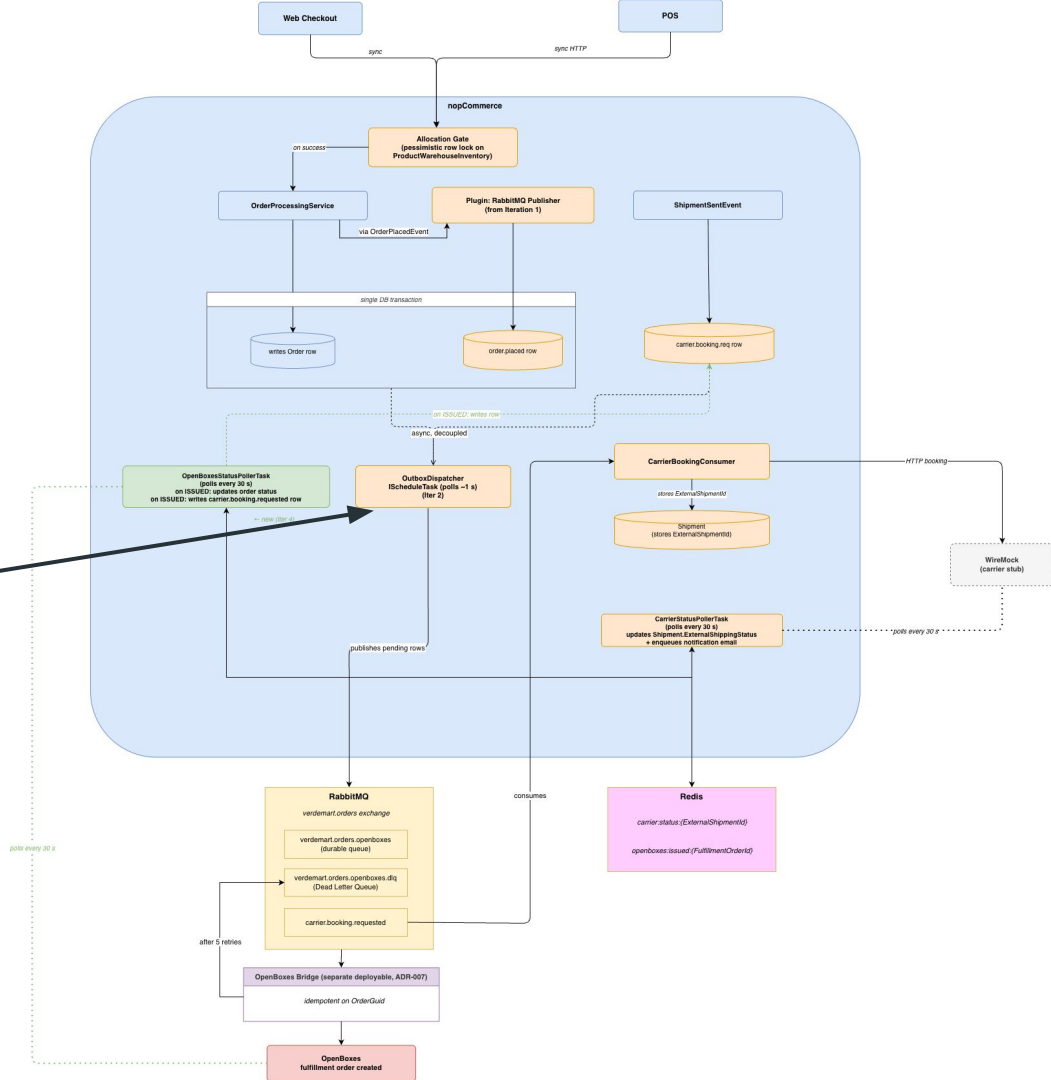
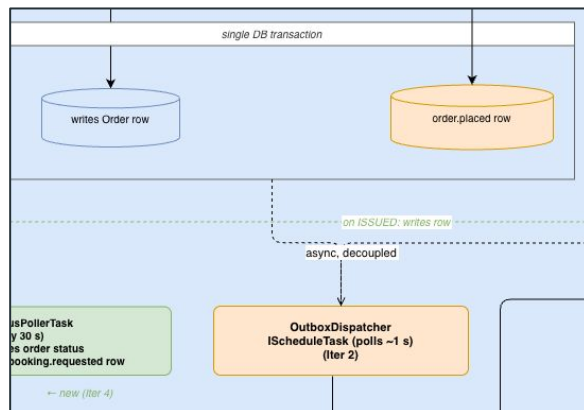
QAS-3: Availability Checkout under 3s regardless of surrounding system latency

QAS-4: Recoverability Missed events must self-heal after outage



Iteration 2

QAS-1: Reliability Zero orders lost during OpenBoxes outage



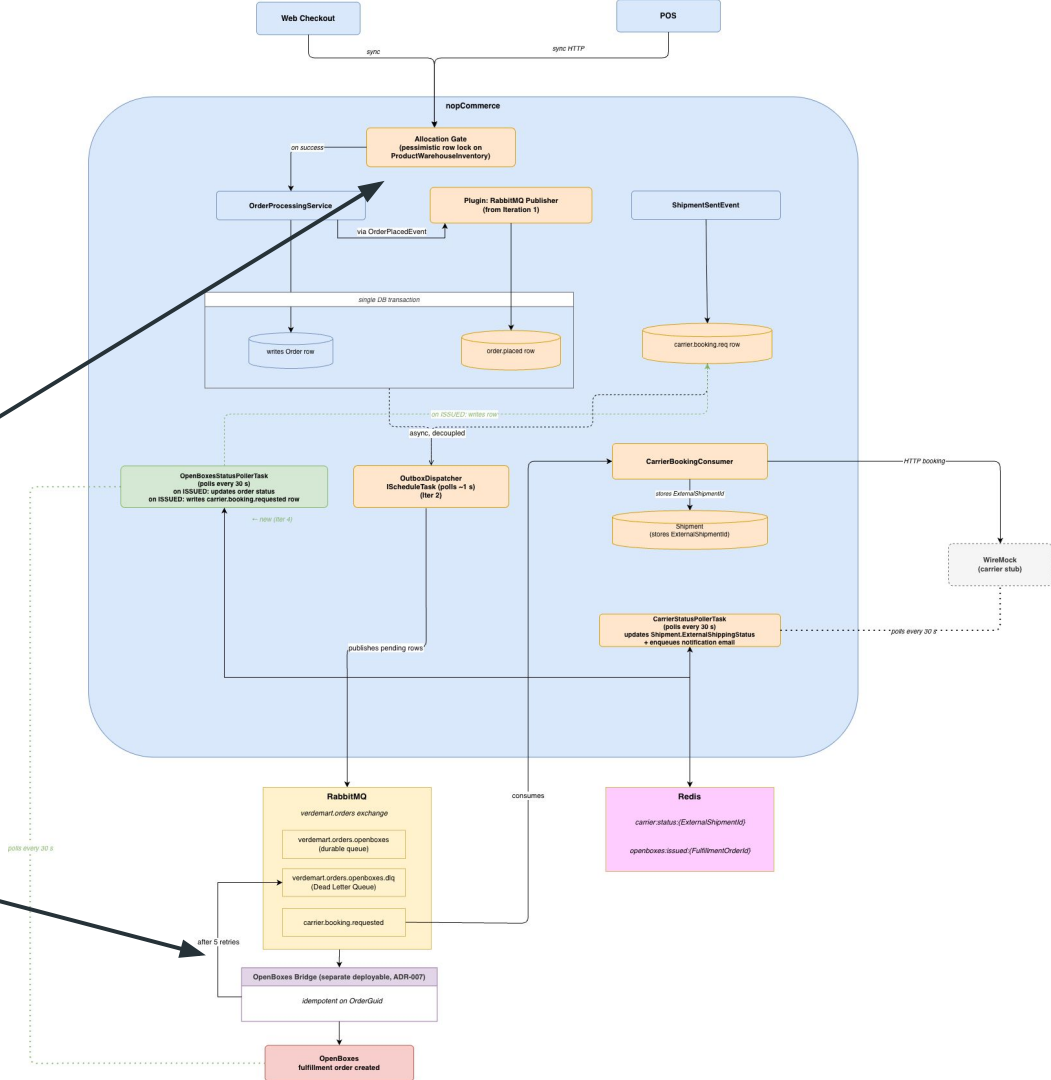
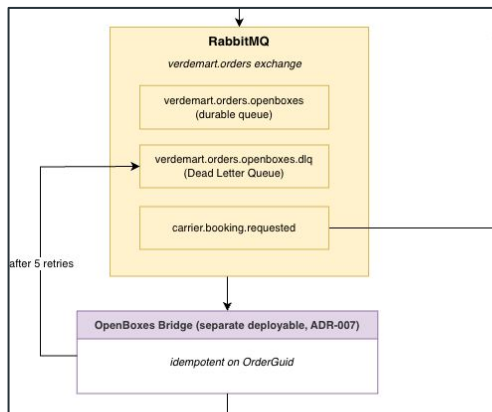
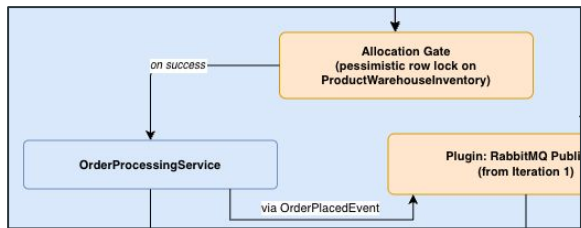
Iteration 3

QAS-2: Consistency

Zero oversell under concurrent web + POS load

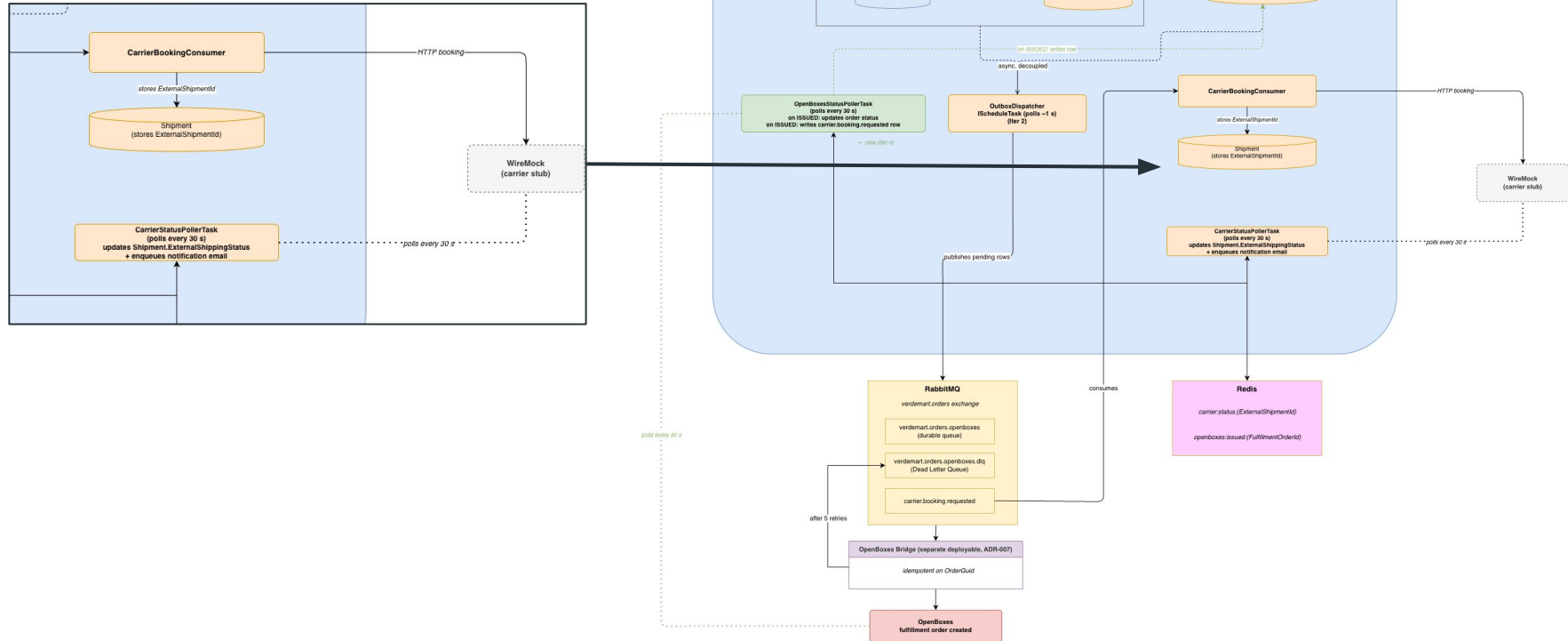
QAS-1: Reliability

QAS-3: Availability Checkout under 3s regardless of surrounding system latency



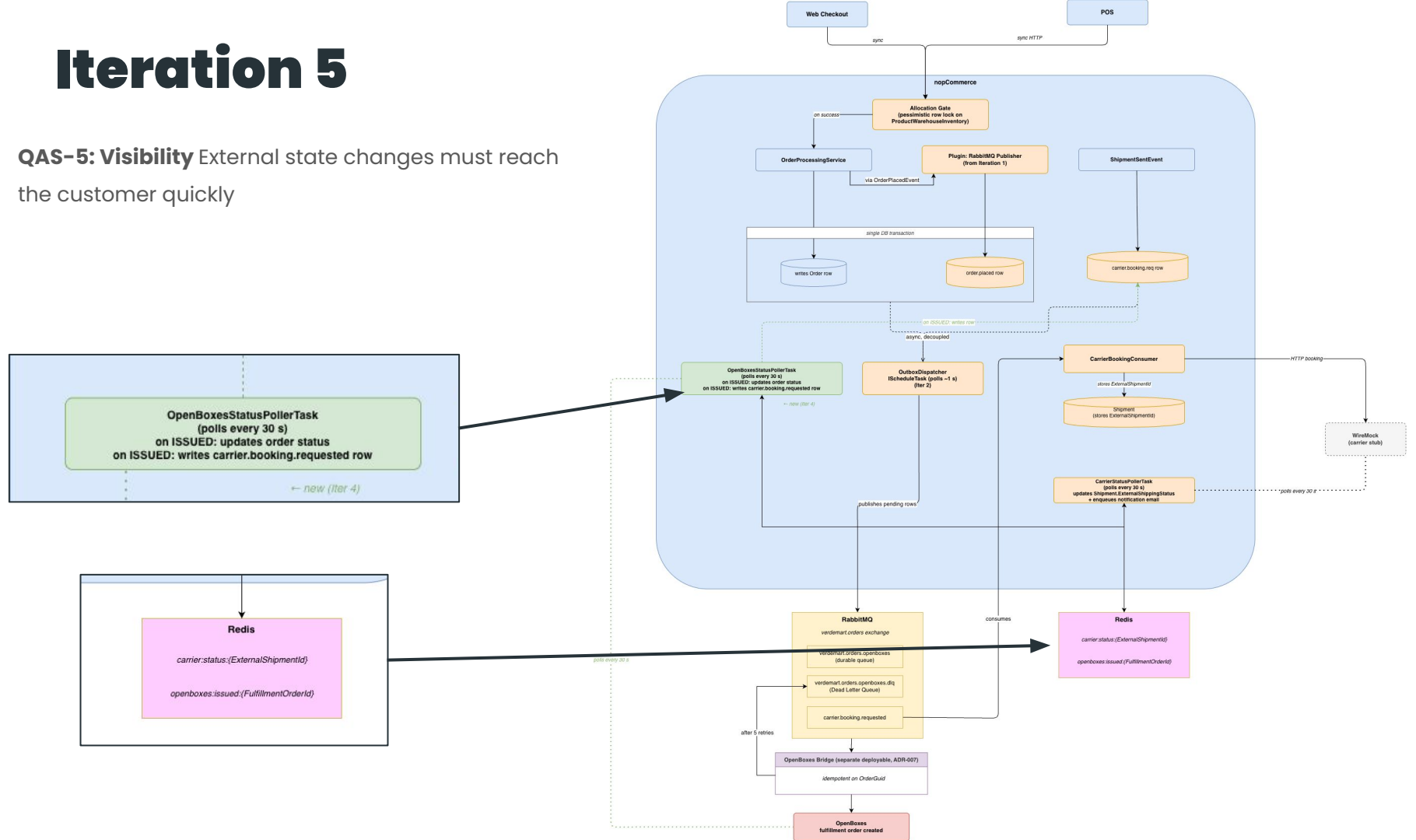
Iteration 4

QAS-5: Visibility External state changes must reach the customer quickly



Iteration 5

QAS-5: Visibility External state changes must reach the customer quickly



Risk & Validation Plan

Risks closed by design

- Order event lost when broker is down → transactional outbox (ADR-004)
- Duplicate fulfillment orders on redelivery → dedup table keyed on OrderGuid
- Concurrent oversell across channels → pessimistic row lock (ADR-005)

Risks still open

- Dispatcher single point of failure - accepted at VerdeMart's scale
- Process-local mutex - safe for single-node demo, known gap at scale
- OpenBoxes API contract - to be confirmed before final demo

Feasibility Spike

```
davidpelicano@davidpelicano-IdeaPad-Gaming-3-15ACH6:~/Desktop/nopCommerce/spike$ docker compose up -d
[+] up 1/1
✓ Container spike-rabbitmq Started
davidpelicano@davidpelicano-IdeaPad-Gaming-3-15ACH6:~/Desktop/nopCommerce/spike$ cd OutboxSpike/
davidpelicano@davidpelicano-IdeaPad-Gaming-3-15ACH6:~/Desktop/nopCommerce/spike/OutboxSpike$ dotnet run
```

=== SCENARIO 1: Happy Path ===

```
[T+0.07s] INSERT order-s1-5d1a4d09 (Status=Pending)
[T+0.15s] Published order-s1-5d1a4d09 → verdemart.orders | Status=Sent
Success: QAS-1 (happy path): row published and marked Sent within 2s
```

=== SCENARIO 2: Broker Down → Recovery ===

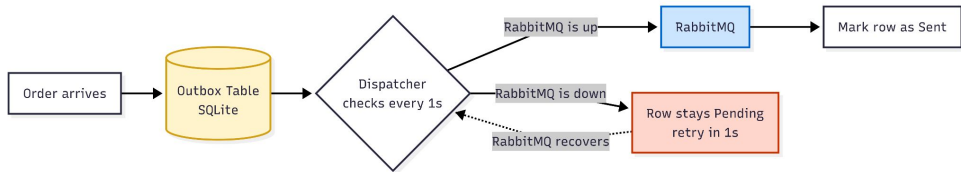
```
[T+0.00s] Stopping spike-rabbitmq...
[T+1.31s] spike-rabbitmq stopped
[T+1.32s] INSERT order-s2-01-190c10 (Status=Pending) → broker is DOWN
[T+1.32s] INSERT order-s2-02-0daa2c (Status=Pending) → broker is DOWN
[T+1.32s] INSERT order-s2-03-9376b1 (Status=Pending) → broker is DOWN
[T+1.32s] Broker unreachable - 3 row(s) remain Pending
[T+2.32s] Broker unreachable - 3 row(s) remain Pending
[T+3.32s] Broker unreachable - 3 row(s) remain Pending
[T+4.32s] Broker unreachable - 3 row(s) remain Pending
[T+5.32s] After 4s with broker DOWN: 3 row(s) still Pending (expected 3)
[T+5.32s] Starting spike-rabbitmq...
[T+5.32s] Broker unreachable - 3 row(s) remain Pending
[T+5.48s] Waiting for broker to become healthy...
[T+6.33s] Broker unreachable - 3 row(s) remain Pending
[T+7.33s] Broker unreachable - 3 row(s) remain Pending
[T+8.33s] Broker unreachable - 3 row(s) remain Pending
[T+9.33s] Broker unreachable - 3 row(s) remain Pending
[T+9.50s] spike-rabbitmq healthy
[T+10.34s] Published order-s2-01-190c10 → verdemart.orders | Status=Sent
[T+10.34s] Published order-s2-02-0daa2c → verdemart.orders | Status=Sent
[T+10.34s] Published order-s2-03-9376b1 → verdemart.orders | Status=Sent
Success: QAS-1 (broker-down): all 3 rows recovered and published after broker restart
```

=== SCENARIO 3: Process Crash Simulation ===

```
[T+0.02s] INSERT order-s3-01-a3c819 (Status=Pending) → dispatcher NOT started (simulates crash)
[T+0.02s] INSERT order-s3-02-cd062d (Status=Pending) → dispatcher NOT started (simulates crash)
[T+0.02s] Fresh dispatcher started (simulates process restart)
[T+0.02s] Published order-s3-01-a3c819 → verdemart.orders | Status=Sent
[T+0.03s] Published order-s3-02-cd062d → verdemart.orders | Status=Sent
Success: QAS-1 (crash recovery): both rows survived and were published after restart
```

All scenarios complete. spike-rabbitmq left running.

```
davidpelicano@davidpelicano-IdeaPad-Gaming-3-15ACH6:~/Desktop/nopCommerce/spike/OutboxSpike$
```



Conclusion

What is decided

- Architecture fully designed across 5 iterations of ADD
- 8 ADRs recorded with rejected alternatives
- All 5 QAS have a named mechanism and a traceable decision

What is validated

- Transactional outbox proven: happy path, broker down, process crash
- Zero message loss confirmed

What comes next - Part 2

- Implement the 5 phases
- Live demonstration under pressure
- Evidence pack with measurements and known limitations

Architectural Evolution of nopCommerce

David Pelicano ◉ Henrique Ferreira ◉ João Monteiro ◉ Jorge Domingues

05.05.2026

